GESTION DES RECRUTEURS

BTS SIO

Réalisé

Au sein de la société TRDEV07

Romain BOUCHARD

Sommaire

Introduction 2
PRESENTATION DE LA SOCIETE TRDEV07 2
Présentation des missions confiées 3
Analyse du Système d'Information et analyse des besoins 4
Création de la base de données et structuration 4
BILAN de l'APPLICATION 32
ORGANISATION 35
CONCLUSION 36
ANNEXES 37

Introduction

Au cours de ma formation de BTS SIO option SLAM, j'ai effectué un stage de 5 semaines (du 18/05/2026 au 19/06/2026) au sein de la société TRDEV07.

Son créateur est régulièrement contacté par des recruteurs pour lui proposer des missions de développement. Il m'a demandé de développer une application lui permettant de gérer tous ces messages.

PRESENTATION DE LA SOCIETE TRDEV07

Nom de l'organisme d'accueil (Raison Sociale) : TRDEV07

Adresse : 594 Chemin d'argevillières, 07000 Privas

Téléphone : 07 67 51 20 22

Forme juridique : SASU

Nombre de salariés : 1

Nom du tuteur : RICARD Thierry

Fonction du tuteur : Créateur de l'entreprise – développeur d'applications

Description de l'activité de l'organisme :

Conseil et services en systèmes et logiciels informatiques ; programmation informatique ; achat, revente, dépannage, réparation et maintenance de matériel informatique ainsi que maintenance logicielle ; formation non réglementée et toutes activités connexes ou liées.

L'entreprise développe SAP en langage ABAP. Son client principal est l'Occitane en Provence en vue de répondre à diverses problématiques liées à la gestion de leur site, des besoins de gestion humaine et autres. Elle peut également intervenir pour d'autres clients.

Description du profil des clients :

La majorité des clients ont des besoins en développement et en conseils informatique.

Description de la place de l'informatique dans l'organisme :

L'entreprise dispose d'un parc informatique composé de deux postes de travail équipés de deux écrans. Elle ne possède pas de serveur. L'ensemble de l'infrastructure est géré par un unique collaborateur qui cumule les fonctions de développeur, de chef d'entreprise et de salarié.

Infrastructure réseau :

L'entreprise utilise uniquement la connectivité fournie par une box domestique, sans solution de pare-feu dédiée (type pfSense ou autre pare-feu matériel).

Accès distant :

Pour ses activités de développement sur SAP (ABAP), l'entreprise établit une connexion sécurisée via un VPN afin d'accéder au réseau du client principal « Occitane en Provence ».

Historique de l'organisme :

Après trente années de salariat, dont huit années passées dans l'administration et vingt et une années au sein de la société Solystic, Thierry Ricard a choisi de se lancer dans l'entrepreneuriat, suite à un plan social fin 2020.

De 2021 à 2022, il a d'abord testé la viabilité de son projet au sein d'une coopérative d'entrepreneurs (Pollen Scop). Cette période d'un an lui a permis de confirmer la solidité de son activité grâce à une facturation régulière, diversifiée entre le dépannage informatique, le développement et le conseil.

Début 2022, une opportunité de collaboration avec la société « Occitane en Provence » l'a conduit à quitter la coopérative pour créer sa propre structure, une SASU, le 1er avril 2022. Depuis cette date, il poursuit sa collaboration avec l'Occitane en Provence tout en développant sa clientèle, notamment pour des prestations de création de sites web et de développement de macros Excel.

« Le seul à avoir eu un Sinclair ZX81 »

THIERRY RICARD, originaire de Buis-les-Baronnies et âgé de 52 ans, s'est installé le 1er avril 2021 comme développeur informatique, à Privas. Pas vraiment une reconversion car ce Drômois, passionné de triathlon, était déjà au cœur de cette activité qui le passionne depuis son plus jeune âge.

Comment avez-vous découvert l'informatique ?
Thierry Ricard. Totalement par hasard. J'avais 12 ans à l'époque et j'ai eu entre mes mains un livre qui traitait de ce sujet. J'ai alors demandé des explications à quelqu'un de féru d'informatique. Du coup, j'ai été convaincu et mes parents m'ont offert un ordinateur Sinclair ZX81. J'étais sans doute le seul de Buis-les-Baronnies à en avoir un. À part peut-être le collège. (rires)

Quel a été votre parcours ensuite ?
T.R. Je suis arrivé à Privas en 1993 et j'ai commencé à travailler à la Caisse primaire d'assurance-maladie comme informaticien, évidemment. Puis, j'ai été développeur à partir de 2001 chez Solystic, à Bourg-les-Valence, spécialisée dans la fourniture de solutions à automatiser de tri et de préparation de la distribution pour les colis et le courrier. J'y suis resté jusqu'au 31 mars 2021 date à laquelle j'ai saisi l'opportunité de me mettre à mon compte à la faveur d'un plan de sauvegarde de l'emploi (Pse). C'est là que j'ai créé TRDEV 07.

Comment êtes-vous passé de salarié à travailleur indépendant ?
T.R. C'était un peu l'inconnue, c'est vrai. En fait, je me suis tourné vers Pollen, une coopérative d'activité et d'entrepreneurs située en Ardèche méridionale, qui m'a accompagné, notamment pour toutes les tâches administratives et comptables. Aujourd'hui, je travaille pour des entreprises qui souhaitent développer des applications, qui n'ont pas forcément de service dédié à ce titre et qui ont besoin de conseils, d'expertise. Mon souhait serait surtout de me diversifier sur le bassin de Privas tout en m'adressant aux particuliers pour des dépannages mais aussi pour des formations.

Par M.L.

Thierry Ricard est fasciné par la science-fiction : « Les auteurs de films et de livres sont souvent en avance sur leur époque. C'est très étonnant, sans doute en lien aussi avec l'informatique. » Photo M.L.

Présentation des missions confiées

Monsieur Ricard travaille à son compte dans la société TRDEV07. Il est spécialiste de l'ERP SAP et développe des programmes spécifiques en ABAP qui s'intègre dans cet ERP.

Il reçoit beaucoup de sollicitations de la part de chefs d'entreprise sur Linkedin qui souhaitent externaliser leur gestion du Recrutement, de leur Logistique, de leur Comptabilité et de leurs services informatiques.

Il est sollicité par des recruteurs car peu de candidats postulent sur le marché du travail avec des compétences spécifiques ABAP. Or la demande des entreprises est importante dans ce domaine. Il reçoit en effet des demandes par mail, par téléphone mais aussi des messages sur son compte professionnel LinkedIn.

Actuellement il emploie 95 % de son temps pour travailler pour « l'Occitane » en Provence et 5% de son temps pour satisfaire des clients proches de Privas.
Il souhaite assurer sa pérennité et faire face à d'autres demandes s'il perdait son client actuel. Il souhaite donc conserver une trace des différents contacts et messages d'entreprises.

Afin de mieux gérer ces sollicitations et garantir la sauvegarde de ces données avant leur traitement, il m'a confié 2 missions.

La première : créer une base de données permettant de mémoriser tous les contacts de propositions de missions.

La deuxième : développer une application en PHP permettant d'administrer la base, de saisir les données, de faire des recherches et présenter un tableau de bord.

Première mission :

Créer une base de données permettant de mémoriser tous les contacts de propositions de missions.

Analyse du Système d'Information et analyse des besoins

Monsieur RICARD souhaite que de nombreuses informations apparaissent pour le traitement des données.

Pour chaque message, il veut retrouver :

- Le numéro du message
- La date du message
- L'identifiant du recruteur (Id)
- Le Message du recruteur
- La Source du message
- Le statut du contact
- La date de relance au recruteur
- Le message de réponse
- Le type du poste
- Le type de travail
- Le lieu du poste (indication géographique)

Pour chaque recruteur, il veut retrouver :

- L'Identifiant du recruteur
- Le nom du recruteur
- L'email du recruteur
- Le Numéro de Téléphone du recruteur
- Id de l'entreprise

Pour l'entreprise il veut retrouver :

- L'identifiant de l'entreprise
- La raison sociale de l'entreprise

Après analyse de ses besoins, j'ai pu créer la base de données et la structurer.

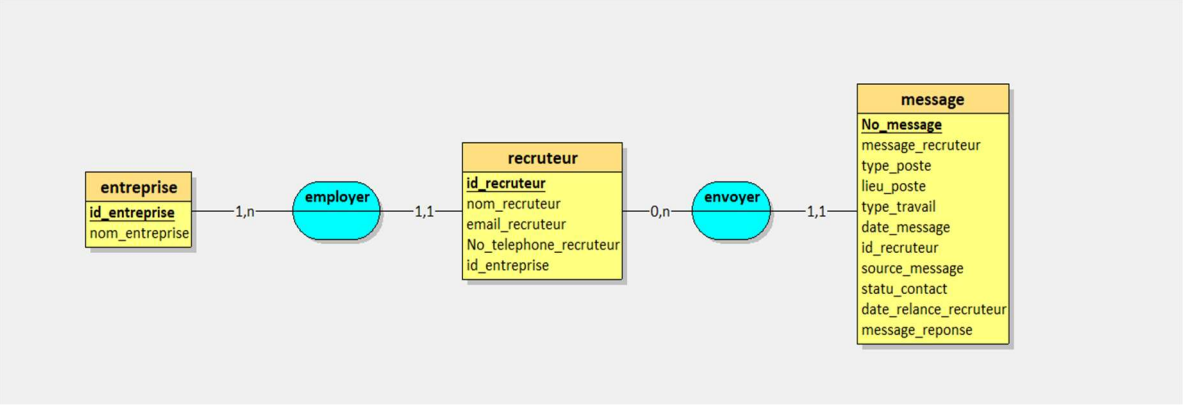
Création de la base de données et structuration

Mon travail s'est effectué en 5 étapes

1ère étape : l'analyse de tous les liens existants entre les informations à mémoriser dans la base de données et les règles de gestion à élaborer

2ème étape : création de 6 règles de gestion nécessaires à Mr Ricard

- Une entreprise peut employer un ou plusieurs recruteurs



- Un recruteur travaille pour une et une seule entreprise
- id_recruteur → Id_entreprise

- Un recruteur envoie de 0 à plusieurs messages
- Un message est envoyé par un et un seul recruteur
- No_message →Id_recruteur

3ème étape : création du modèle conceptuel de données pour répondre à ses besoins

Une fois les règles de gestion établies, j'ai pu concevoir le MCD sous Looping, créer 3 entités :

. l'entité Entreprise, . l'entité Recruteur, . l'entité Message

Une fois cette étape terminée, un script a pu être généré ce qui a permis de créer la Base de Données répondant à ces besoins.

```
CREATE TABLE entreprise(
   id_entreprise INT,
   nom_entreprise VARCHAR(200),
   CONSTRAINT PK_entreprise PRIMARY KEY(id_entreprise)
);

CREATE TABLE recruteur(
   id_recruteur INT,
   nom_recruteur VARCHAR(100),
   email_recruteur VARCHAR(150),
   No_telephone_recruteur VARCHAR(20),
   id_entreprise INT,
   id_entreprise INT NOT NULL,
   CONSTRAINT PK_recruteur PRIMARY KEY(id_recruteur),
   CONSTRAINT FK_recruteur_entreprise_1 FOREIGN
KEY(id_entreprise_1) REFERENCES entreprise(id_entreprise)
);

CREATE TABLE message(
   No_message INT,
```

```
   message_recruteur VARCHAR(2000),
   type_poste VARCHAR(50),
   lieu_poste VARCHAR(100),
   type_travail VARCHAR(50),
   date_message DATE,
   id_recruteur INT,
   source_message VARCHAR(50),
   statu_contact VARCHAR(50),
   date_relance_recruteur DATE,
   message_reponse VARCHAR(2000),
   id_recruteur INT NOT NULL,
   CONSTRAINT PK_message PRIMARY KEY(No_message),
   CONSTRAINT FK_message_recruteur_1 FOREIGN KEY(id_recruteur_1)
REFERENCES recruteur(id_recruteur)
);
```

4 étape : tests sur la base de données structurée en vue d'une modification ultérieure par un tiers ou en mon absence ou pour répondre à d'autres besoins

Après analyse et essais, j'ai dû modifier les 3 tables de la Base de Données.

En effet, les identifiants n'étaient pas auto-incrémentés dans le script généré, ce qui aurait généré des problèmes d'utilisation.

5ème étape : dans cette dernière étape, j'ai corrigé le script généré en modifiant le nom des colonne référencées en clés étrangères. (J'ai enlevé le _1 ajouté par Looping).

J'ai modifié le type des identifiant qui était INT, pour permettre l'auto-incrémentations. L'interface PHPMyAdmin, permet de cocher la case auto-increment associée à la colonne des identifiants.

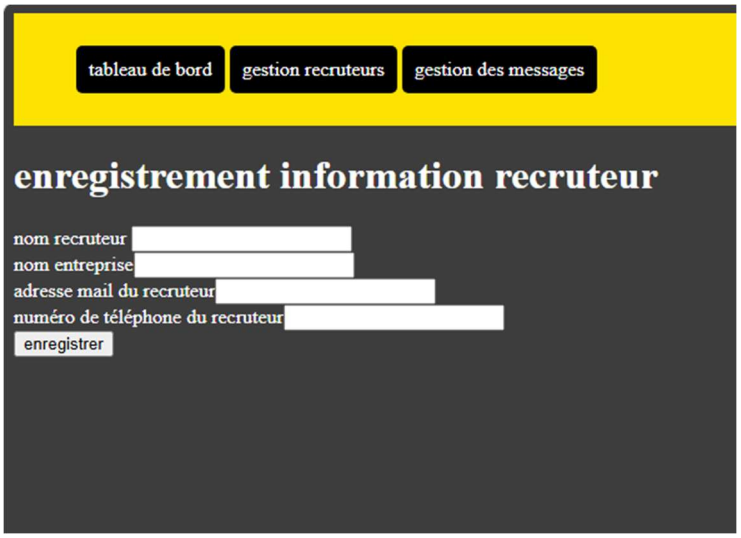
Deuxième mission : Développer une application « Gestion Recruteurs » en PHP permettant d'administrer la base, de saisir les données, de faire des recherches et présenter un tableau de bord.

Pour atteindre les objectifs de cette mission j'ai procédé encore une fois par étapes.

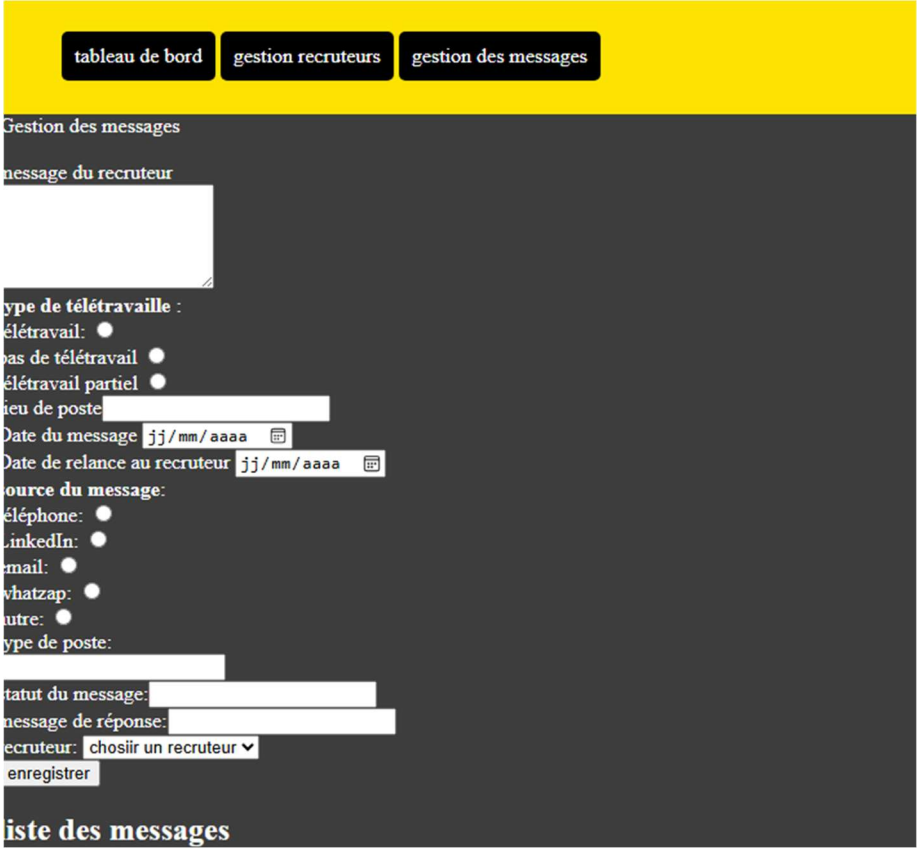
1ère étape : réaliser 3 maquettes des différentes pages de l'application PHP

La maquette d'un tableau de bord

La maquette Gestion Recruteurs



La maquette Gestion des messages



2ème étape :

Une fois les fondations posées, j'ai pu commencer à gérer les interactions entre le PHP et la base de données.

J'ai commencé par écrire un script PHP permettant de se connecter à la base de données.

```
<?php
function connexionbdd()
{
   $env = parse_ini_file('.env');
   $host = $env['DB_HOST'];
   $dbname = $env['DB_NAME'];
   $userbd = $env['DB_USER'];
   $passwordbd = $env['DB_PASS'];
   try
   {
      $conexion = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8", $userbd,
$passwordbd);
      // On configure PDO pour qu'il affiche les futures erreurs SQL s'il y en a
      $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
//      echo "Connexion à la base de données réussie !";
      return $conexion;
   } catch (PDOException $e)
   {
      die('Erreur : impossible de se connecter à la base de donnée' . $e->getMessage());
   }
}
```

J'ai ensuite créé un script permettant d'insérer des valeurs pour les entreprises et les recruteurs dans la section "gestion des recruteurs" .

```
<?php
require_once('bdd.php');
if (isset($_POST['valider'])) {
    $nom_entreprise = $_POST['nom_entreprise'];
    $nom_recruteur = $_POST['nom_recruteur'];
    $adresse_mail_recruteur = $_POST['email_recruteur'];
    $num_tel_recruteur = $_POST['tel_recruteur'];
    $bdd = connexionbdd();
    $reqverifyentreprise = $bdd->prepare("SELECT * FROM entreprise WHERE nom_entreprise =
:nom_entreprise");
    $reqverifyentreprise->execute(array('nom_entreprise' => $nom_entreprise));
    $nom_entrepriserequresult = $reqverifyentreprise->fetch(PDO::FETCH_ASSOC);
    $id_entreprise = $nom_entrepriserequresult['id_entreprise'];
    if ($nom_entrepriserequresult === false)
    {
        $reqinsertentreprise = $bdd->prepare("INSERT INTO entreprise (nom_entreprise)
VALUES (:nom_entreprise)");
        $reqinsertentreprise->execute(array('nom_entreprise' => $nom_entreprise));
        $id_entreprise = $bdd->lastInsertId();
    }

    $reqinsertrecruteur = $bdd->prepare("INSERT INTO recruteur
(nom_recruteur,email_recruteur,No_téléphone_recruteur,id_entreprise)
                                        VALUES
(:nom_recruteur,:email_recruteur,:tel_recruteur,:id_entreprise)");
    $reqinsertrecruteur -> execute(array(
        ':nom_recruteur' => $nom_recruteur,
        ':email_recruteur' => $adresse_mail_recruteur,
        ':tel_recruteur' => $num_tel_recruteur,
        ':id_entreprise' => $id_entreprise
    ));

    header("Location: enregistremnt_information_recruteur.php?success");
    exit();

}
if (isset($_GET['success'])):?>
<p>Enregistré avec succès</p>

<?php endif; ?>
```

e stade, la base de données est bien structurée.

3ème étape : Tests fonctionnels du script d'insertion. J'ai effectué plusieurs essais d'insertion de données. A la fin de cette étape, je vérifiais sous PHPMyAdmin que les données ajoutées dans l'interface, étaient bien présentes dans les tables de la base de données.

4ème étape : la gestion des messages

J'ai ensuite réalisé un script permettant de saisir des messages et de les mémoriser dans la base.

Ci-dessous, je présente un extrait de ce script.

```
$messageconfirm =  "<p>message enregistre avec succes.</p>";
if(isset($_POST['valider']))
{
    $message = $_POST['message'];
    $teletravail = $_POST['teletravail'];
    $date = $_POST['date'];
    $source = $_POST['source'];

    if($source === 'autre') {
        $source = $_POST['autre_inputtext'];
```

```
    }
    $type_poste = $_POST['type_poste'];
    $date_relance_recruteur = $_POST['date_relance_recruteur'];
    $message_reponsse = $_POST['message_reponsse'];
    $statut_contact = $_POST['statu_contact'];
    $lieu_poste = $_POST['lieu_poste'];
    $id_recruteur = $_POST['id_recruteur'];
    $reqInsertMessage = $bdd->prepare("INSERT INTO message (message_recruteur,
source_message, statu_contact, date_relance_recruteur, message_reponse,

type_poste,lieu_poste,type_travail,date_message,id_recruteur)
                                    VALUES (:message, :source,
:statut_contact,:date_relance_recruteur,:message_reponse,
                                            :type_poste,:lieu_poste,:teletravail,
:date_message, :id_recruteur)");

    if($reqInsertMessage->execute(array(
        ':message' => $message,
        ':teletravail' => $teletravail,
        ':date_message' => $date,
        ':source' => $source,
        ':type_poste' => $type_poste,
        ':date_relance_recruteur' => $date_relance_recruteur,
        ':message_reponse' => $message_reponsse,
        ':statut_contact' => $statut_contact,
        ':lieu_poste' => $lieu_poste,
        ':id_recruteur' => $id_recruteur
    ))) {
        header('Location: gestion_messages.php');
        echo $messageconfirm;
    } else {
        echo "<p>Erreur lors de l'enregistrement du message.</p>";
    }
}
```

Une fois ce script fonctionnel, j'ai réalisé plusieurs saisies de messages.

5ème étape : l'affichage des messages enregistrés

J'ai donc affiché les messages dans l'interface de « Gestion des messages », en y intégrant des boutons pour éditer et supprimer.

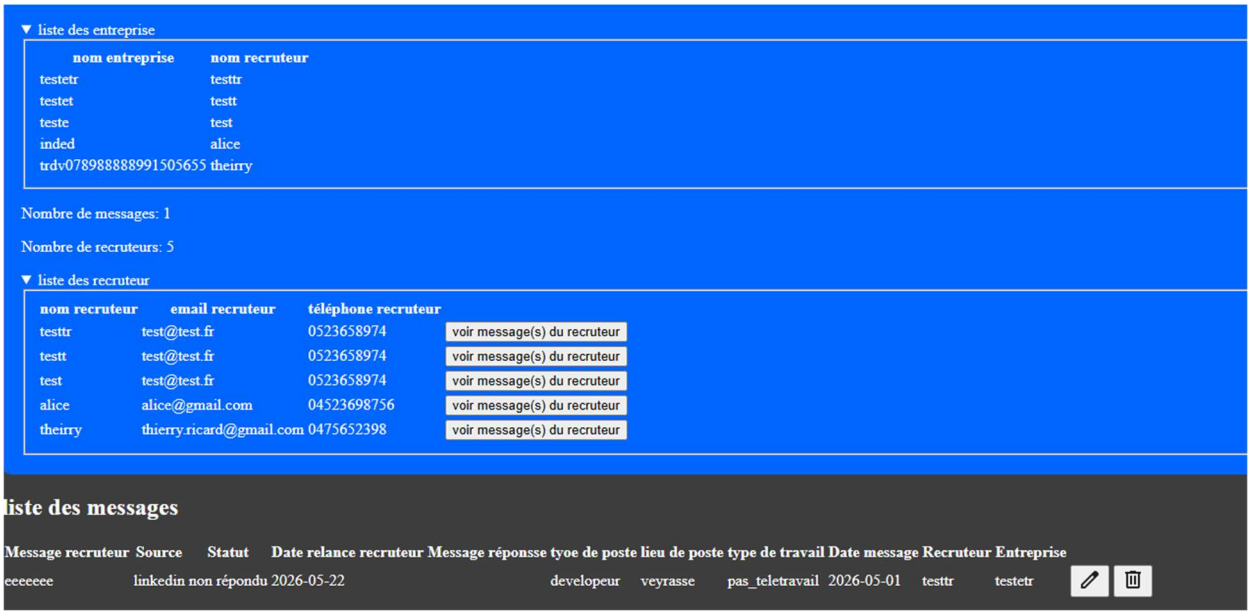
liste des messages

Message recruteur	Source	Statut	Date relance recruteur	Message réponsse	tyoe de poste	lieu de poste	type de travail	Date message	Recruteur	Entreprise		
fffff gggg hhh jjj	telephone	non répondu	2026-05-22		developeur	veyrasse	teletravail	2026-05-01	theirry	trdv078988888991505655		
dddd dddd ddd ffff	linkedin	non répondu	2026-05-22		developeur	veyrasse	pas_teletravail	2026-05-01	theirry	trdv078988888991505655		
dddddd	whatsap	non répondu	2026-05-22		developeur	veyrasse	pas_teletravail	2026-05-01	alice	inded		

```
<?php foreach ($messageliste as $listemessage): ?>
    <tr>
        <td><?= htmlspecialchars($listemessage['message_recruteur']) ?></td>
        <td><?= htmlspecialchars($listemessage['source_message']) ?></td>
        <td><?= htmlspecialchars($listemessage['statu_contact']) ?></td>
        <td><?= htmlspecialchars($listemessage['date_relance_recruteur'])?></td>
        <td><?= htmlspecialchars($listemessage['message_reponse']) ?></td>
        <td><?= htmlspecialchars($listemessage['type_poste']) ?></td>
        <td><?= htmlspecialchars($listemessage['lieu_poste']) ?></td>
        <td><?= htmlspecialchars($listemessage['type_travail']) ?></td>
        <td><?= htmlspecialchars($listemessage['date_message']) ?></td>
        <td><?= htmlspecialchars($listemessage['nom_recruteur']) ?></td>
        <td><?= htmlspecialchars($listemessage['nom_entreprise']) ?></td>
        <td><button type="button" name="modify" value="<?=$listemessage['No_message']?>"><img
src="assets/edit.svg" alt="modify"></button></td>
        <td><button type="button" name="delete" value="<?=$listemessage['No_message']?>"><img
src="assets/delete.svg" alt="delete"></button></td>
    <?php endforeach; ?>
    </tr>
    </table>
</main>
```

Lorsque j'ai obtenu un affichage des messages conforme à mes attentes, j'ai programmé le code associé aux boutons "modifier" et "supprimer".

J'ai ensuite mis en place un filtre par recruteur lors du clic sur le bouton "voir messages du recruteur" dans la liste des recruteurs. Cette option permet de visualiser tous les messages concernant le recruteur sélectionné.



Ci-dessous, la capture d'écran présente le formulaire de modification des informations d'un message. En bas d'écran on a une liste de messages. Le bouton Modifier (pictogramme crayon) permet d'afficher les informations détaillées du message dans la partie supérieure de l'écran. L'utilisateur peut modifier les informations puis cliquer sur 'Enregistrer' pour les mémoriser dans la base.

Le script suivant gère l'affichage du panneau de bord contenant le nombre d'entreprises, de messages et de recruteurs.

```php
include 'traitement_ajax.php';
require_once('bdd.php');
$bdd = connexionbdd();
$reqcountentreprise = $bdd->prepare( query: "select count(*) from entreprise");
$reqcountentreprise->execute();
$rescountentrepriseresult = $reqcountentreprise->fetch( mode: PDO::FETCH_ASSOC)['count(*)'];

$reqcountmessage = $bdd->prepare( query: "select count(*) from message");
$reqcountmessage->execute();
$rescountmessageresult = $reqcountmessage->fetch( mode: PDO::FETCH_ASSOC)['count(*)'];

$reqcountrecruteur = $bdd->prepare( query: "select count(*) from recruteur");
$reqcountrecruteur->execute();
$rescountrecruteurrestult = $reqcountrecruteur->fetch( mode: PDO::FETCH_ASSOC)['count(*)'];
?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <link rel="stylesheet" href="style/style.css">
    <title>accueil</title>
</head>
<body>
<header>
    <?php include "header.php";
    ?>
</header>
<main>
<h1>tableau de bord</h1>
<div id="tableau_de_bord">
    <h2>tableau de bord</h2>
    <?php
    if(isset($bdd))
        {
    echo "<p> Nombre d'entreprises: ". $rescountentrepriseresult ."</p>";
```

L’ extrait du code suivant permet d’afficher la liste des message avec le filtre par recruteur

```
document.querySelectorAll('.voir_recruteur').forEach(button
=> {
    button.addEventListener('click', function() {
        fetch('liste_message.php', {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-
form-urlencoded' },
            body: 'action=filter&id_recruteur=' +
this.value
        })
        .then(response => response.json())
        .then(messages => {
            const tableBody =
document.querySelector('#form_message tbody') ||
                document.querySelector('#form_message');
            let list_message = `
            <tr>
                <th>Message recruteur</th>
                <th>Source</th>
                <th>Statut</th>
                <th>Date relance recruteur</th>
                <th>Message réponse</th>
                <th>Type de poste</th>
                <th>Lieu de poste</th>
                <th>Type de travail</th>
                <th>Date message</th>
                <th>Recruteur</th>
                <th>Entreprise</th>
                <th> </th>
                <th> </th>
            </tr>`;
            messages.forEach(msg => {
                list_message += `
                <tr>
                    <td>${msg.message_recruteur || ''}</td>
                    <td>${msg.source_message || ''}</td>
                    <td>${msg.statu_contact || ''}</td>
                    <td>${msg.date_relance_recruteur ||
''}</td>
                    <td>${msg.message_reponse || ''}</td>
                    <td>${msg.type_poste || ''}</td>
                    <td>${msg.lieu_poste || ''}</td>
                    <td>${msg.type_travail || ''}</td>
                    <td>${msg.date_message || ''}</td>
                    <td>${msg.nom_recruteur || ''}</td>
                    <td>${msg.nom_entreprise || ''}</td>
```

```
function lierEvenementsBoutons() {
    document.querySelectorAll('.modify').forEach(button =>
{
        button.addEventListener('click', function() {
            fetch(window.location.href, {
                method: 'POST',
                headers: { 'Content-Type': 'application/x-
www-form-urlencoded' },
                body: 'action=modify&id=' + this.value
            })
            .then(response => response.json())
            .then(data => {
                const msg =
document.getElementById('message');
                if (msg) msg.value =
data.message_recruteur;
            });
        });
    });

    document.querySelectorAll('.delete').forEach(button =>
{
        button.addEventListener('click', function() {
            if (confirm('Voulez-vous vraiment supprimer ce
message ?')) {
                fetch(window.location.href, {
                    method: 'POST',
                    headers: { 'Content-Type':
'application/x-www-form-urlencoded' },
                    body: 'action=delete&id=' + this.value
                })
                .then(() => {
                    window.location.reload();
                });
            }
        });
    });
}
lierEvenementsBoutons();
```

```
<h2>liste des messages</h2>
<?php
require_once ('bdd.php');
$bdd = connexionbdd();
if (isset($_POST['action']) && $_POST['action'] === 'filter') {
    $idrecruteur = $_POST['id_recruteur'];
    $selectmessage = "SELECT message_recruteur, source_message,
statu_contact, date_relance_recruteur,
                      message_reponse, type_poste, lieu_poste,
                      type_travail, date_message,
nom_entreprise, nom_recruteur, No_message
                      FROM message
                      INNER JOIN recruteur on
message.id_recruteur = recruteur.id_recruteur
                      INNER JOIN entreprise on
recruteur.id_entreprise = entreprise.id_entreprise";
    $params = array();
    if (!empty($idrecruteur)) {
        $selectmessage .= " WHERE message.id_recruteur =
:id_recruteur";
        $params[':id_recruteur'] = $idrecruteur;
    }
    $reqselectmessage = $bdd->prepare($selectmessage);
    $reqselectmessage->execute($params);
    $messageliste = $reqselectmessage-
>fetchAll(PDO::FETCH_ASSOC);
    ob_clean();
    header('Content-Type: application/json');
    echo json_encode($messageliste);
```

```
    exit;
}
$selectmessage = "SELECT message_recruteur, source_message,
statu_contact, date_relance_recruteur,
                  message_reponse, type_poste, lieu_poste,
                  type_travail, date_message, nom_entreprise,
nom_recruteur, No_message
                  FROM message
                  INNER JOIN recruteur on message.id_recruteur =
recruteur.id_recruteur
                  INNER JOIN entreprise on
recruteur.id_entreprise = entreprise.id_entreprise";

$reqselectmessage = $bdd->prepare($selectmessage);
$reqselectmessage->execute();
$messageliste = $reqselectmessage->fetchAll(PDO::FETCH_ASSOC);
?>
<style>#form_message{border="1";}</style>
<table id="form_message">
<tr>
    <th>Message recruteur</th>
    <th>Source</th>
```

Extrait de la liste des recruteurs

```
<?php
require_once ('bdd.php');
$bdd = connexionbdd();
$reqrecruteur = $bdd->prepare("select * from recruteur");
$reqrecruteur->execute();
$reqlistrecruteur = $reqrecruteur->fetchAll(PDO::FETCH_ASSOC);
?>
<fieldset>
<table>
    <th>nom recruteur</th>
    <th>email recruteur</th>
    <th>téléphone recruteur</th>
    <th></th>
    <?php
    foreach ($reqlistrecruteur as $listrecruteur)
    {
        ?>
        <tr>
            <td><?=
htmlspecialchars($listrecruteur['nom_recruteur']) ?></td>
            <td><?=
htmlspecialchars($listrecruteur['email_recruteur']) ?></td>
            <td><?=
htmlspecialchars($listrecruteur['No_téléphone_recruteur']) ?></td>
            <td><button class="voir_recruteur" value="<?=
$listrecruteur['id_recruteur'] ?>">voir message(s) du
recruteur</button></td>
        </tr>
    <?php
    }
    ?>
</table>
</fieldset>
```

Extrait de la liste des entreprises

```
<?php
require_once ('bdd.php');
$bdd = connexionbdd();
$reqentreprise = $bdd->prepare("select nom_entreprise,nom_recruteur from entreprise
                                inner join recruteur on entreprise.id_entreprise =
recruteur.id_entreprise");
$reqentreprise->execute();
$reqlistentreprise = $reqentreprise->fetchAll(PDO::FETCH_ASSOC);
?>
<fieldset>
    <table>
        <th>nom entreprise</th>
        <th>nom recruteur</th>
        <?php
        foreach ($reqlistentreprise as $listentreprise)
        {
            ?>
            <tr>
                <td><?= htmlspecialchars($listentreprise['nom_entreprise']) ?></td>
                <td><?= htmlspecialchars($listentreprise['nom_recruteur']) ?></td>
            </tr>
            <?php
        }
        ?>
    </table>
</fieldset>
```

J'ai ensuite fait en sorte qu'on puisse modifier un message directement depuis le tableau de bord. Cela se traduit par une redirection, transmettant l'identifiant du message, vers la page de gestion des messages où le formulaire se remplit automatiquement à l'arrivée. Pour réaliser cette fonctionnalité, j'ai mis en place un nouvel écouteur d'événement en JavaScript sur la page gestion_messages.php

```
if (isset($_POST['action']) && $_POST['action'] === 'modify' &&
isset($_POST['id'])) {
    $id_message = $_POST['id'];
    $req = $bdd->prepare("SELECT * FROM message WHERE No_message =
:id");
    $req->execute(array(':id' => $id_message));
    $messageData = $req->fetch(PDO::FETCH_ASSOC);

    if ($messageData) {
        ob_clean();
        header('Content-Type: application/json');
        echo json_encode($messageData);
        exit;
    }
}
$reqselectrecruteur = $bdd->prepare("SELECT * FROM recruteur");
```

Pour le pré-remplissage du formulaire, j'ai adapté la logique qui n'était initialement présente que sur la page de gestion des messages. Désormais, ce processus se déclenche si l'URL contient l'identifiant du message, ce qui permet de modifier un message que l'on soit dans la page 'Gestion des messages' ou dans la page 'Tableau de bord'.

```
+if (action === 'modify' && idMessage) {
    fetch('gestion_messages.php', {
        method: 'POST',
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: 'action=modify&id=' + idMessage
    })
    .then(response => response.json())
    .then(data => {
        const msg = document.getElementById('message');
        const lieu = document.getElementById('lieu_poste');
        const type = document.getElementById('type_poste');
        const stat = document.getElementById('statu_message');
        const rep = document.getElementById('message_reponse');
        const dateMsg = document.getElementById('date');
        const dateRel = document.getElementById('date_relance');
        const selectRec = document.getElementById('select_recruteur');

        if (msg) msg.value = data.message_recruteur;
        if (lieu) lieu.value = data.lieu_poste;
        if (type) type.value = data.type_poste;
        if (stat) stat.value = data.statu_contact;
        if (rep) rep.value = data.message_reponse;
        if (dateMsg) dateMsg.value = data.date_message;
        if (dateRel) dateRel.value = data.date_relance_recruteur;
        if (selectRec) selectRec.value = data.id_recruteur;

        const inputId = document.getElementById('id_message');
        if (inputId) inputId.value = data.No_message;

        if (data.type_travail) {
            const radioTravail = document.getElementById(data.type_travail);
            if (radioTravail) radioTravail.checked = true;
        }

        const sources = ['telephone', 'linkedin', 'email', 'whatsap'];
        const inputAutre = document.getElementById('autreInput');

        if (inputAutre) {
            inputAutre.style.display = 'none';
```

J'ai ensuite ajouté un filtre sur la liste des entreprises afin de n'afficher sur le tableau de bord que les messages liés à une entreprise spécifique. Pour ce faire, j'ai dû encapsuler la logique du fichier modification.js dans une fonction, évitant ainsi tout conflit avec le filtre des recruteurs.

```
Show usages   new *
function remplirLesChampsDuFormulaire(data) :void {
```

```
function lierEvenementsBoutons() : void {
    document.querySelectorAll( selectors: '.modify').forEach( callbackfn: button : Element => {
        button.addEventListener( type: 'click', listener: function() : void {
            if (window.location.pathname.includes('index.php')) {
                window.location.href = 'gestion_messages.php?action=modify&id=' + this.value;
                return;
            }
            fetch(window.location.href, init: {
                method: 'POST',
                headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
                body: 'action=modify&id=' + this.value
            }) Promise<Response>
                .then(response : Response => response.json()) Promise<any>
                .then(data => {
                    remplirLesChampsDuFormulaire(data);
                });
        });
    });
```

```
lierEvenementsBoutons();

const urlParams : URLSearchParams = new URLSearchParams(window.location.search);
const action : string = urlParams.get('action');
const idMessage : string = urlParams.get('id');

if (action === 'modify' && idMessage) {
    fetch(window.location.href, init: {
        method: 'POST',
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: 'action=modify&id=' + idMessage
    }) Promise<Response>
        .then(response : Response => response.json()) Promise<any>
        .then(data => {
            remplirLesChampsDuFormulaire(data);
        }) Promise<void>
        .catch(error => console.error("Erreur lors du pré-remplissage automatique :", error));
}
```

```
<?php
require_once ('bdd.php');
$bdd = connexionbdd();
$reqentreprise = $bdd->prepare("select nom_entreprise,
entreprise.id_entreprise from entreprise");
$reqentreprise->execute();
$reqlistentreprise = $reqentreprise->fetchAll(PDO::FETCH_ASSOC);
?>
<fieldset>
    <table>
        <th>nom entreprise</th>
        <th> </th>
        <?php
        foreach ($reqlistentreprise as $listentreprise)
        {
            ?>
            <tr>
                <td><?=
htmlspecialchars($listentreprise['nom_entreprise']) ?></td>
                <td><button class="voir_entreprise" value="<?=
$listentreprise['id_entreprise']?>">voir message(s) de l'
            </tr>
            <?php
        }
        ?>
    </table>
</fieldset>
```

J’ai ensuite commencé à configurer WampServer afin d'autoriser l’accès au site depuis le réseau de l’entreprise. Pour cela, j’ai modifié le Virtual host que j’avais créé initialement pour accéder plus rapidement au site en local grâce au Server Name. J’y ai ajouté un Server Alias permettant ainsi, même sans modification préalable du fichier hosts pour lier le nom de domaine à l'adresse IP, d'accéder directement à l'application via l’adresse IP du serveur (le PC).

```
<VirtualHost *:80>
  ServerName gestion_recruteur.trdev07
  ServerAlias 192.168.0.73
  DocumentRoot "c:/wamp64/www/gestion_recruteur"
  <Directory "c:/wamp64/www/gestion_recruteur">
    AllowOverride All
    Require all granted
  </Directory>
</VirtualHost>
```

J’ai également dû m’assurer que le fichier de configuration d'Apache (httpd.conf) possédait bien les ports d'écoute nécessaires à l'ouverture de l'accès sur le réseau

```
Listen 0.0.0.0:80
Listen [::0]:80
```

J’ai ensuite commencé à concevoir un jeu d'essai. Avant d'intégrer les données en base de données, j'ai créé un fichier Excel afin de documenter les enregistrements. Cela permet de disposer d'une sauvegarde de sécurité en cas de dysfonctionnement ou de problème technique sur la base de données

No_message	message_recruteur	source_message	statu_contact	date_relance_recruteur	message_reponse	type_poste	lieu_poste	t
	Nous recherchons un consultant SAP DMS pour un client dans le secteur aéronautique. Contexte Dans le cadre d'un vaste programme de transformation digitale, un acteur majeur du secteur aéronautique poursuit le décommissionnement des solutions Microsoft au profit d'environnements plus modernes et collaboratifs. Certaines dépendances subsistent toutefois, notamment via le module SAP DMS, nécessitant une sortie progressive de ces	email	non répondu			consultant SAP-DMS	Toulouse + p	t
		linkedin	non répondu					
	Nous avons une mission de Lead Dev ABAP à proposer chez l'un de nos clients finaux SAP. Encadrement de plusieurs développeurs ABAP. 2 jours de télétravail, 3 jours sur site en province. Je suis disponible pour en discuter. Vous pouvez également me renseignez vos attentes pour un nouvelle mission de freelance SAP (cliquez sur le lien) Ou me préciser votre	email	non répondu			developeur ABAP	Province	t

A	B	C	D
nom_recruteur	email_recruteur	No_Téléphone_recruteur	Id_entreprise

A	B
nom_entreprise	
alternative-search	
silkhom	
signeplus	
redglobal	
washingtonfrank	

En attendant mon tuteur pour résoudre un problème, j'ai ensuite travaillé sur le CSS et ajouté au tableau de bord un graphique représentant les messages répondus et non répondus.

```
<?php
$pourcentage = 0;
if ($rescountmessageresult > 0) {
    $pourcentage = ($rescountmessagereponduresult / $rescountmessageresult) * 100;
}
?>

<div class="kpi_container">

    <div class="cercle_ratio" style="--pourcent: <?= $pourcentage ?>%;">
        <div class="cercle_centre">
            <?= $rescountmessagereponduresult ?> / <?= $rescountmessageresult ?>
        </div>
    </div>

    <div class="kpi_legende">
        <div class="legende_item">
            <span class="carre_couleur couleur_repondu"></span>
            <span class="texte_legende">Messages répondus (<?= $rescountmessagereponduresult
?>)</span>
        </div>
        <div class="legende_item">
            <span class="carre_couleur couleur_non_repondu"></span>
            <span class="texte_legende">Messages non répondus (<?= ($rescountmessageresult -
$rescountmessagereponduresult) ?>)</span>
        </div>
    </div>

</div>
```

```
$reqcountmessagerepondu = $bdd->prepare("select count(*) from
message where message_reponse is not null and message_reponse
!= ''");
```

```
$reqcountmessagerepondu->execute();

$rescountmessagereponduresult = $reqcountmessagerepondu-
>fetchColumn();
```

Le passage à fetchColumn() a permis l'affichage du graphique en évitant de récupérer les données sous forme de tableau, ce qui aurait bloqué sa génération.

Pour ce premier jour de télétravail, j'ai pu continuer à mettre à jour le jeu d'essai dans Excel pour la table recruteur. Étant à distance, je me suis rendu compte de l'utilité d'un tel jeu d'essai sur Excel, car il m'a permis de réinsérer des valeurs très rapidement.

J'avais initialement rencontré une erreur d'insertion en créant mon jeu d'essai pour les recruteurs : j'avais mis toutes les valeurs dans les mêmes guillemets (ou apostrophes), alors qu'il fallait des

fx insert into recruteur (nom_recruteur,email_recruteur,No_Téléphone_recruteur,Id_entreprise)

	A	B	C	D	
1	nom_recruteur	email_recruteur	No_Téléphone_recruteur	Id_entreprise	insert into recruteur (nom_recruteur,email_recruteur,No_Tél
2	**Nina Couderchon**	n.couderchon@washingtonfrank.com	+33 1 59 14 13 31	5	insert into recruteur (nom_recruteur,email_recruteur,No_Tél
3	**Fanny SICARD**	fanny.sicard@signeplus.com		3	insert into recruteur (nom_recruteur,email_recruteur,No_Tél
4	**Jennifer Isaksen**	jisaksen@redglobal.com	+44 (0) 207 107 7611	4	insert into recruteur (nom_recruteur,email_recruteur,No_Tél
5	**Jibril Salmi**	j.salmi@silkhom.com	06 73 84 69 72	2	insert into recruteur (nom_recruteur,email_recruteur,No_Tél
6	**Alexandre Bonin**	abonin@alternative-search.com	+33 7 57 91 88 21	1	insert into recruteur (nom_recruteur,email_recruteur,No_Tél
7	**Daniël Kalvenhaar**	invitations@linkedin.com		6	insert into recruteur (nom_recruteur,email_recruteur,No_Tél

guillemets distincts pour chaque valeur.="insert into recruteur (" & A1&"," & B1 & ","& C1&"," & D1 & ") value ('" & A1&"','" &B1&"','"& C1 &"','"& D1 & ");"

J'ai terminé le jeu d'essai pour les messages. À cette occasion, j'ai rencontré des problèmes d'échappement des apostrophes dans le texte des messages : il a fallu doubler l'apostrophe (écrire '' au lieu de ') pour des mots comme "l'entreprise" (devenant ainsi "l''entreprise"). Sans cela, l'apostrophe fermait prématurément la chaîne de caractères de la requête SQL, ce qui générait une erreur de syntaxe.

J'ai également rencontré un problème avec le format des dates. J'ai dû réécrire les dates dans Excel après avoir modifié le format des cellules en "Texte", car le format "Date" initial d'Excel ne correspondait pas au format attendu par la base de données.

Enfin, une erreur s'est produite dans la requête d'insertion en raison d'une ligne où je n'avais pas renseigné l'identifiant du recruteur (id_recruteur). La requête a échoué car cette colonne, étant une clé étrangère (et donc non-auto-incrémentée), requiert obligatoirement une valeur valide.

insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes
insert into message (message_recruteur,source_message,stat	insert into message (message_recruteur,source_mes

Une fois toutes les données insérées, j'ai dû tout supprimer et réinsérer proprement. En effet, j'avais oublié de réinitialiser l'auto-incrémentation, ce qui générait des identifiants erronés par rapport à ceux définis dans Excel, où j'étais parti du principe que la numérotation commençait à 1.

Une fois ce problème corrigé, le site affichait correctement le résultat des requêtes. Cependant, les messages apparaissaient tronqués. Cela venait du fait que j'avais défini le type de la colonne des messages en VARCHAR(200) dans la base de données, ce qui s'est révélé insuffisant pour stocker l'intégralité du texte de certains messages.

Erreur de requête:
#1118 - Ligne trop grande. Le taille maximale d'une ligne, sauf les BLOBs, est 65535. Changez le type de quelques colonnes en BLOB

Pour corriger ce problème, j'ai modifié le type du champ concerné pour le passer en BLOB (ou TEXT. Cela résout le problème de limite du texte enregistré.

Par ailleurs, j'ai ajouté un bouton "Voir plus" en bas du tableau afin de ne pas afficher la totalité des messages d'un coup. J'ai ainsi mis en place une limitation à 10 messages affichés par page écran. Cette fonctionnalité a été développée en PHP et en JavaScript (via une requête AJAX).

Du côté du JavaScript, j'ai dû modifier l'écouteur d'événement existant afin de rendre ce bouton "Voir plus" pleinement fonctionnel.

```
?>
    <tr id="ligne_voir_plus">
        <td colspan="13">
            <button type="button" id="btn_voir_plus" data-offset="10">Voir plus</button>
        </td>
    </tr>
</table>
```

En PHP, j'ai donc ajouté le bouton "Voir plus" en lui associant un attribut data-offset="10", qui correspond au nombre de lignes que le bouton doit charger lors du clic.

```
else if (isset($_POST['action']) && $_POST['action'] === 'load_more') {
    $offset = isset($_POST['offset']) ? (int)$_POST['offset'] : 0;

    // On récupère les 10 messages suivants grâce à OFFSET
    $selectmessage = "SELECT message_recruteur, source_message, statu_contact, date_relance_recruteur,
                            message_reponse, type_poste, lieu_poste,
                            type_travail, date_message, nom_entreprise, nom_recruteur, No_message
                      FROM message
                      INNER JOIN recruteur on message.id_recruteur = recruteur.id_recruteur
                      INNER JOIN entreprise on recruteur.id_entreprise = entreprise.id_entreprise
                      LIMIT 10 OFFSET :offset"; // Limite à 10 à partir de l'offset donné

    $reqselectmessage = $bdd->prepare($selectmessage);
    $reqselectmessage->bindValue( param: ':offset', $offset,  type: PDO::PARAM_INT);
    $reqselectmessage->execute();
    $messageliste = $reqselectmessage->fetchAll( mode: PDO::FETCH_ASSOC);

    ob_clean();
    header( header: 'Content-Type: application/json');
    echo json_encode($messageliste);
    exit;
}
```

J'ai également dû ajouter un bloc else if pour que, si l'utilisateur clique sur le bouton "Voir plus", l'application charge les 10 messages suivants grâce à la clause OFFSET.

Cependant, suite à l'ajout de ce bouton, mes filtres permettant de trier les messages par recruteur ou par entreprise ne fonctionnaient plus. J'ai donc dû déboguer le code à partir du script de traitement AJAX pour identifier l'origine du problème. Cela m'a conduit à analyser les requêtes SQL de la liste des messages : l'erreur venait du fait qu'une requête comportait deux clauses WHERE, et que la clause LIMIT 10 était positionnée avant le WHERE au lieu d'être placée après.

J'ai ajouté la liste des recruteurs dans la section de gestion des recruteurs, en y intégrant la possibilité de les modifier et de les supprimer. Pour cela, j'ai repris la logique développée précédemment pour les messages en l'adaptant aux recruteurs.

J'ai simplement rencontré une difficulté avec le champ "nom de l'entreprise" qui ne se remplissait pas automatiquement. En effet, la table recruteur ne stocke que l'identifiant de l'entreprise (id_entreprise). Il me faut donc réaliser une requête (une jointure SQL) afin de récupérer le nom de l'entreprise correspondant à cet identifiant.

```
if ($_POST['id_recruteur'])
{
    $requpdaterecruteur = $bdd->prepare("update recruteur set
nom_recruteur = :nom_recruteur,id_entreprise =
:id_entreprise,email_recruteur = :email_recruteur,

No_telephone_recruteur = :telephone_recruteur where
id_recruteur = :id_recruteur");
    if ($requpdaterecruteur->execute(array(
        ':nom_recruteur' => $nom_recruteur,
        ':id_entreprise' => $id_entreprise,
        ':email_recruteur' => $adresse_mail_recruteur,
        ':telephone_recruteur' => $num_tel_recruteur,
        ':id_recruteur' => $_POST['id_recruteur']
    )))
```

Pour modifier un recruteur, il m'a fallu faire deux fonctions remplirChampFormulairerecruteur et lierEvenementBouttonsrecruteur

```
function remplirLesChampsDuFormulairerecruteur(data) : void {
    const nomr : HTMLElement = document.getElementById( elementId: 'nom_recruteur');
    const nom                 getElementById( elementId: 'nom_entreprise');
    const ema                 .getElementById( elementId: 'email_recruteur');
    const telr : HTMLElement = document.getElementById( elementId: 'tel_recruteur');

    if (nomr) nomr.value = data.nom_recruteur || '';
    if (nome) nome.value = data.nom_entreprise || '';
    if (emailr) emailr.value = data.email_recruteur || '';
    if (telr) telr.value = data.No_telephone_recruteur || '';
    const inputId : HTMLElement = document.getElementById( elementId: 'id_recruteur');
    if (inputId) inputId.value = data.id_recruteur || '';
}

Show usages  new *
function lierEvenementsBoutonsrecruteur() : void {
    document.querySelectorAll( selectors: '.modifyrecruteur').forEach( callbackfn: button : Element => {
        button.addEventListener( type: 'click', listener: function() : void {
            if (window.location.pathname.includes('index.php')) {
                window.location.href = 'gestion_recruteur.php?action=modify&id=' + this.value;
                return;
            }

            fetch( input: 'gestion_recruteur.php', init: {
                method: 'POST',
```

Pour voir la modification il a fallu que j'affiche une nouvelle fois la liste des recruteurs. Ce code permet d'afficher la liste actualisée si l'action modifier est effectuée.

```php
<?php
require_once('bdd.php');
$bdd = connexionbdd();
if (isset($_POST['action']) && $_POST['action'] === 'modifyrecruteur') {
    $id_recruteur = $_POST['id'];
    $req = $bdd->prepare( query: "SELECT * FROM recruteur WHERE id_recruteur = :id");
    $req->execute(['id' => $id_recruteur]);
    $recruteurData = $req->fetch( mode: PDO::FETCH_ASSOC);
    echo json_encode($recruteurData);
    exit;
}
?>
```

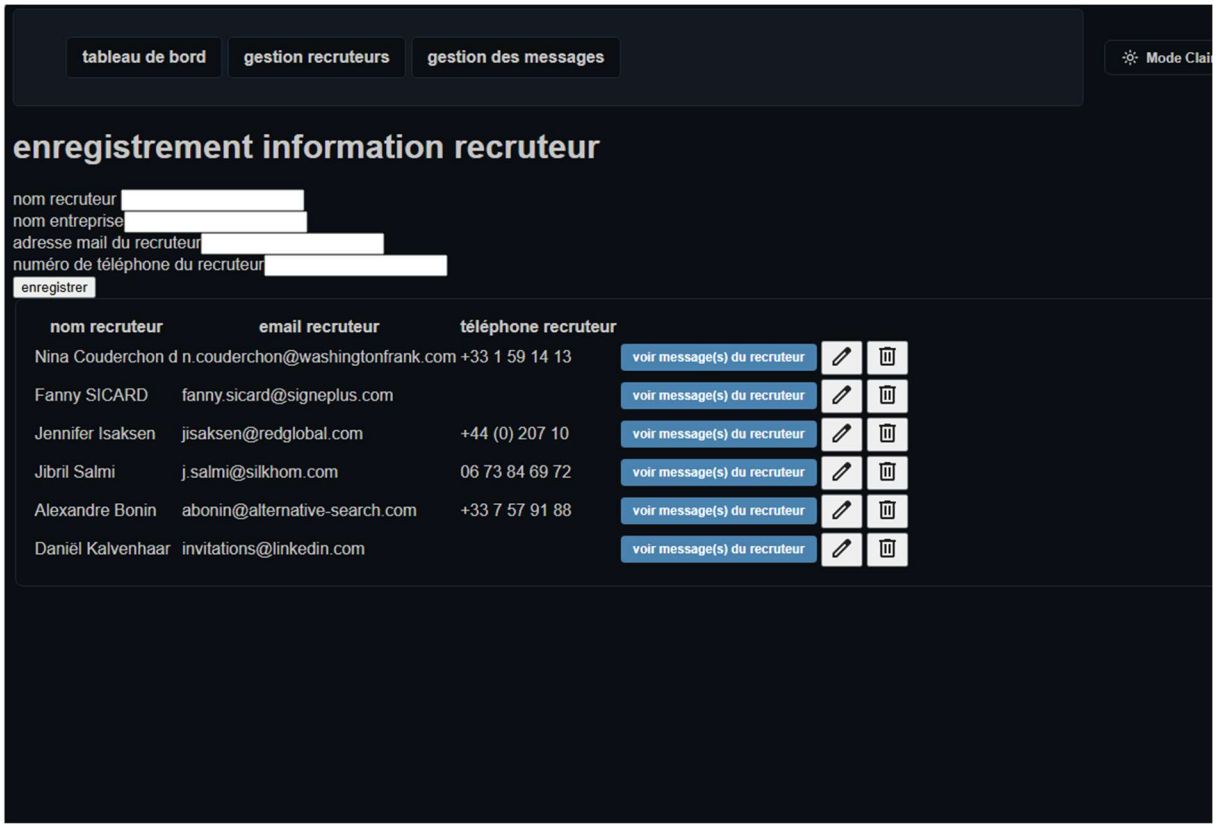
J'ai donc dû mettre en valeur les différentes lignes de la liste des recruteurs sur le bouton modifier id recruteur

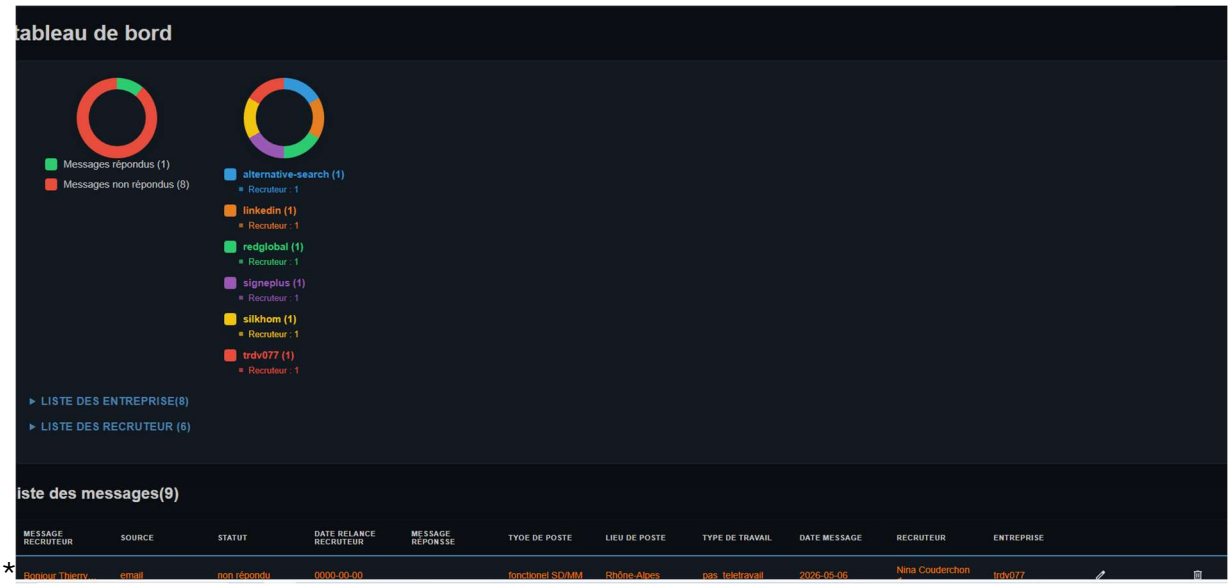
```
<td><?= htmlspecialchars($listrecruteur['email_recruteur']) ?></td>
            <td><?=
htmlspecialchars($listrecruteur['No_telephone_recruteur']) ?></td>
            <td><button class="voir_recruteur" value="<?=
$listrecruteur['id_recruteur']?>">voir message(s) du
recruteur</button></td>
            <td> <button class="modifyrecruteur" value="<?=
$listrecruteur['id_recruteur']?>"><img alt="modifyrecruteur"
src="assets/edit.svg"></button></td>
            <td> <button class="deleterecruteur" value="<?=
$listrecruteur['id_recruteur']?>"><img alt="deleterecruteur"
src="assets/delete.svg"></button></td>
        </tr>
        <?php
    }
    if (isset($_POST['action'])) {
        $id_recruteur = $_POST['id'];
        if ($_POST['action'] === 'deleterecruteur') {
            $texte = "Bouton cliqué ! Action : " . $_POST['action']
. " | No reçu : " . $id_recruteur . " à " .
                    date('H:i:s') . "\n";
            file_put_contents('log.txt', $texte, FILE_APPEND);
            $reqdeletemessage = $bdd->prepare("delete from
recruteur where id_recruteur = :id_recruteur");
            $reqdeletemessage -> execute(array(
                ':id_recruteur' => $id_recruteur
            ));
            header('Location: gestion_recruteur.php');
        }
    }
?><script src="script/listerecruteur.js"></script>
```

J'ai également résolu une erreur liée à paramURL, où il a fallu isoler la variable avec des accolades. De plus, j'ai ajouté un favicon pour ne plus avoir d'erreur affichée dans la console.

```
{
    const paramsUrl : URLSearchParams = new URLSearchParams(window.location.search);
    const actionModif : string = paramsUrl.get('action');
    const idMsg : string = paramsUrl.get('id');

    if (actionModif === 'modify' && idMsg) {
        fetch( input: 'gestion_messages.php', init: {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
            body: 'action=modify&id=' + idMsg
        }) Promise<Response>
            .then(response : Response => response.json()) Promise<any>
            .then(data => {
                remplirLesChampsDuFormulaire(data);
            }) Promise<void>
            .catch(error => console.error("Erreur lors du pré-remplissage :", error));
    }
}
```

J'ai également amélioré le design du site et ajouté un graphique des entreprises par recruteur, afin de visualiser rapidement le nombre de recruteurs par entreprise.





Dans l'amélioration du design j'ai ajouté 2 graphiques pour voir les messages répondus et non répondus et un autre pour voir les recruteurs par entreprise. Voici le css du graphique

```
liste_recruteur.php   liste_message.php   nav.php   gestion_messages.php   graphic.css   header.php

.grille_legende {
    display: grid !important;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
    width: 100%;
    margin-top: 15px;
}

.bloc_entreprise {
    display: flex;
    flex-direction: column;
    gap: 6px;
    align-items: flex-start;
}

.ligne_entreprise {
    display: flex;
    align-items: center;
    gap: 10px;
}

.titre_entreprise {
    font-weight: bold;
    text-transform: none;
}

.liste_recruteurs_bloc {
    margin-left: 22px;
    font-size: 13px;
    display: flex;
    flex-direction: column;
    gap: 4px;
}

.ligne_recruteur {
    display: flex;
    align-items: center;
    gap: 8px;
}

.carre_recruteur {
    width: 6px;
```

Voici le code pour les graphiques

```
// recruteur x entreprise
$reqLegende = $bdd->prepare("
    SELECT e.nom_entreprise, r.nom_recruteur
    FROM entreprise e
    INNER JOIN recruteur r ON e.id_entreprise = r.id_entreprise
    ORDER BY e.nom_entreprise
");
$reqLegende->execute();
$listeLegende = $reqLegende->fetchAll(PDO::FETCH_ASSOC);
$totalCouples = count($listeLegende);
$entreprisesGroupees = [];
foreach ($listeLegende as $ligne) {
    $ent = $ligne['nom_entreprise'];
    if (!isset($entreprisesGroupees[$ent])) {
        $entreprisesGroupees[$ent] = [];
    }
    $entreprisesGroupees[$ent][] = $ligne['nom_recruteur'];
}

$tranchesCc = [];
$currentAngle = 0;
$couleursFixes = ['#3498db', '#e67e22', '#2ecc71', '#9b59b6',
'#f1c40f', '#e74c3c'];
$indexCouleur = 0;

if ($totalCouples > 0) {
    foreach ($entreprisesGroupees as $entreprise => $recruteurs) {
        $couleur = $couleursFixes[$indexCouleur %
count($couleursFixes)];

        $partAngle = (count($recruteurs) / $totalCouples) * 360;
        $nextAngle = $currentAngle + $partAngle;

        $tranchesCc[] = "{$couleur} {$currentAngle}deg
{$nextAngle}deg";
        $currentAngle = $nextAngle;

        $indexCouleur++;
    }
}
$styleConicGradient = !empty($tranchesCc) ? implode(", ",
$tranchesCc) : "#1f2a36";
?>
```

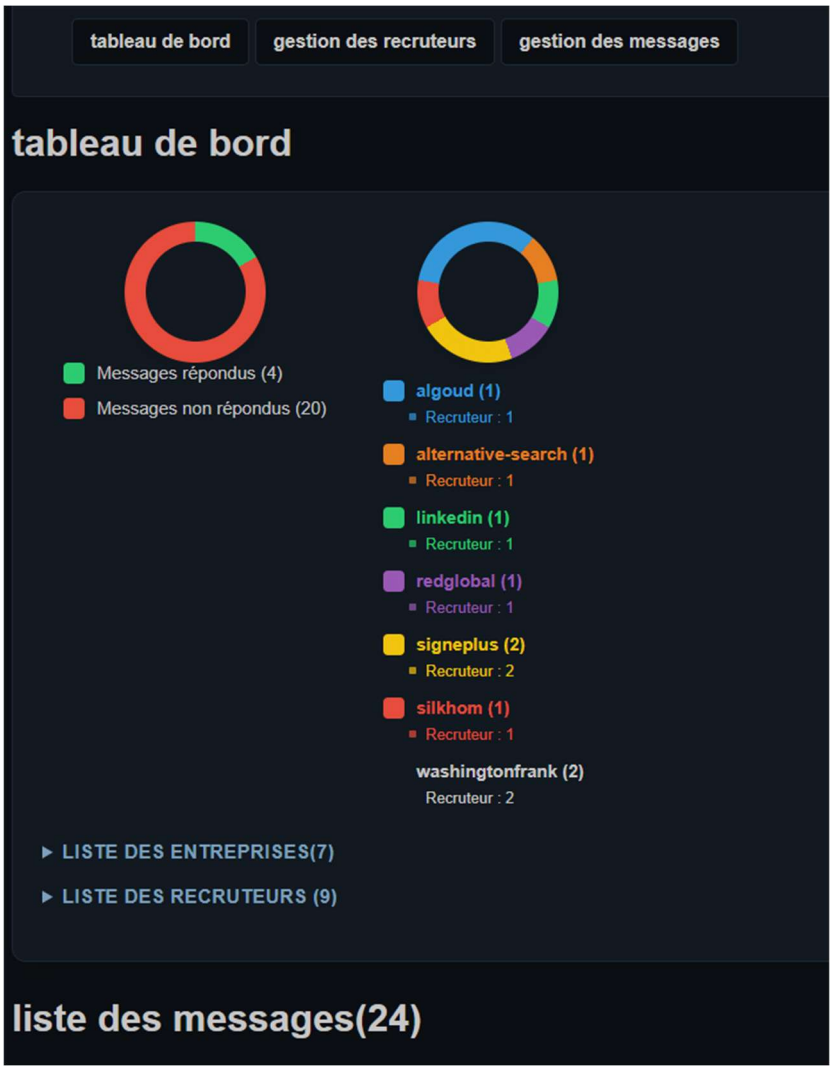
```
<div id="graph-contain">
    <div id="tableau_de_bord">
        <div class="zone_graphiques_globale">

            <div class="colonne_graphique messages">
                <div class="cercle_ratio" style="...">
                    <div class="cercle_centre"></div>
                </div>

                <div class="kpi_legende">
                    <div class="legende_item">
                        <span class="carre_couleur
couleur_repondu"></span>
                        <span class="texte_legende">Messages répondus
(<?= $rescountmessagereponduresult ?>)</span>
                    </div>
                    <div class="legende_item">
                        <span class="carre_couleur
couleur_non_repondu"></span>
                        <span class="texte_legende">Messages non répondus
(<?= ($rescountmessageresult - $rescountmessagereponduresult) ?>)</span>
                    </div>
                </div>
            </div>

            <div class="colonne_graphique entreprises">
                <div class="cercle_ratio" style="...">
                    <div class="cercle_centre"></div>
                </div>

                <div class="grille_legende">
                    <?php foreach ($entreprisesGroupees as $entreprise =>
$recruteurs): ?>
                        <div class="bloc_entreprise">

                            <div class="ligne_entreprise">
                                <span class="carre_couleur"></span>
                                <span class="texte_legende
titre_entreprise">
                                <?= htmlspecialchars($entreprise) ?> (<?=
count($recruteurs) ?>)
                                </span>
                            </div>

                            <div class="liste_recruteurs_bloc">
```

BILAN de l'APPLICATION

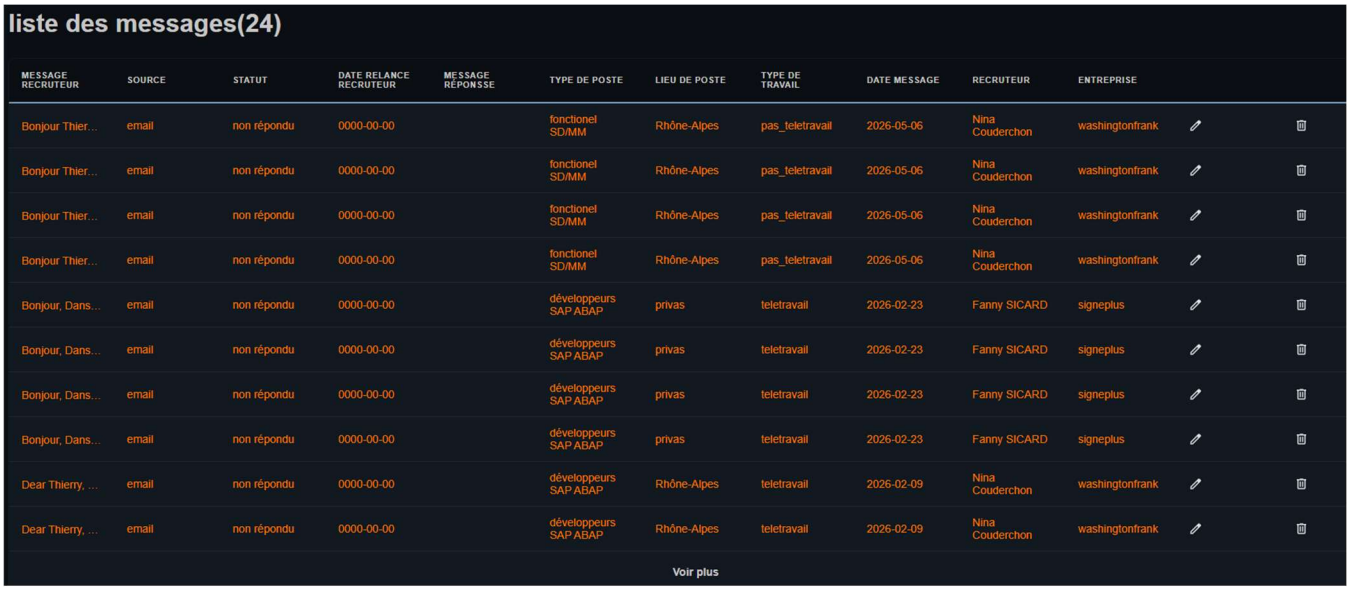
A l'issue des 5 semaines de stage, l'application Gestion des recruteurs est terminée.

Je l'ai utilisée pour saisir plusieurs messages en conditions réelles d'utilisation. Tout le code fonctionne sans problème.

Lorsqu'on lance l'application on obtient l'écran suivant.



Lorsqu'on descend plus bas sur la page, on obtient la liste des 10 premiers messages de la liste. (Le chiffre 24 indique le nombre de messages total mémorisés dans la base.)

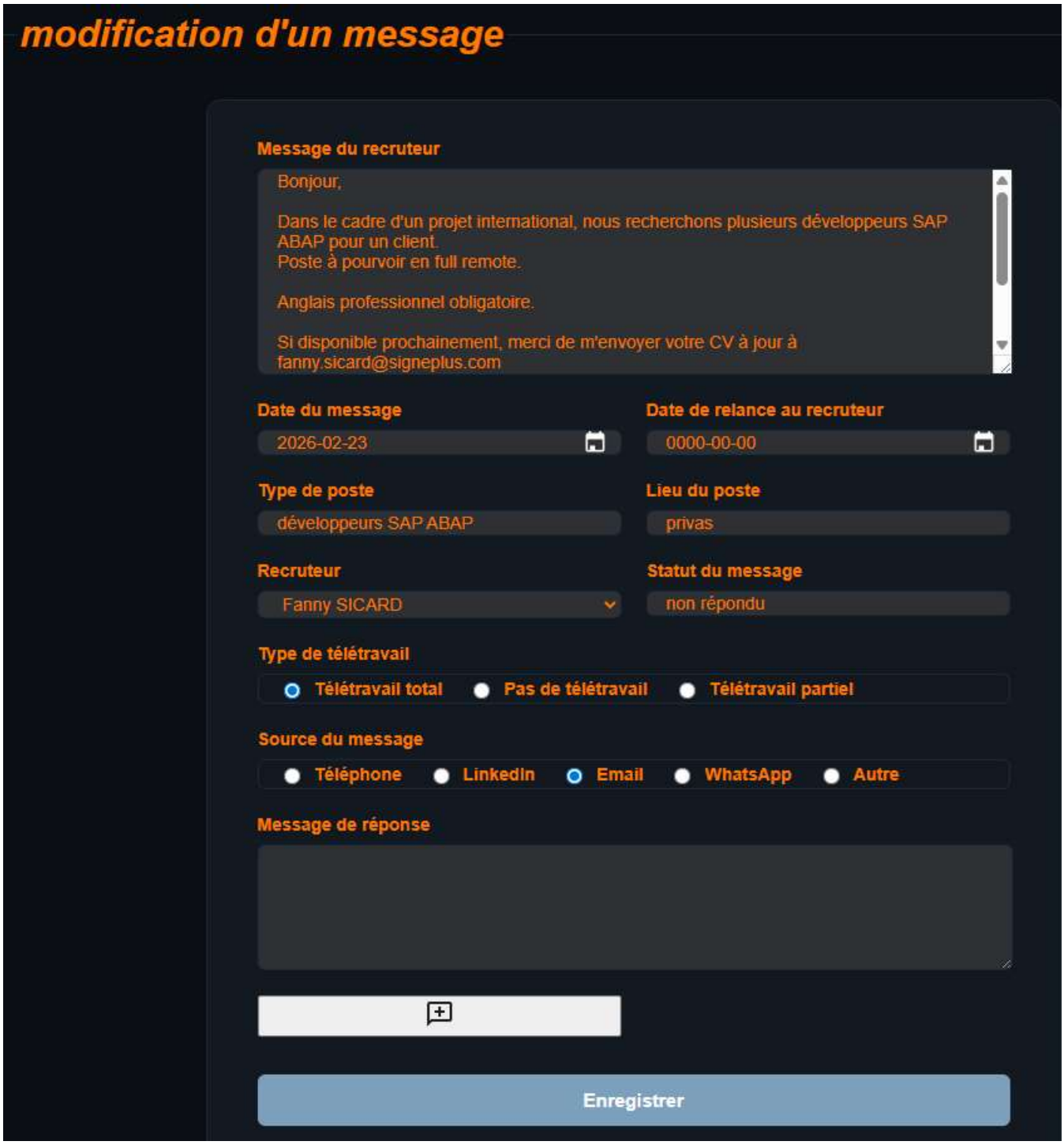


liste des messages(24)

MESSAGE RECRUTEUR	SOURCE	STATUT	DATE RELANCE RECRUTEUR	MESSAGE RÉPONSE	TYPE DE POSTE	LIEU DE POSTE	TYPE DE TRAVAIL	DATE MESSAGE	RECRUTEUR	ENTREPRISE
Bonjour Thier...	email	non répondu	0000-00-00		fonctionel SD/MM	Rhône-Alpes	pas_teletravail	2026-05-06	Nina Couderchon	washingtonfrank
Bonjour Thier...	email	non répondu	0000-00-00		fonctionel SD/MM	Rhône-Alpes	pas_teletravail	2026-05-06	Nina Couderchon	washingtonfrank
Bonjour Thier...	email	non répondu	0000-00-00		fonctionel SD/MM	Rhône-Alpes	pas_teletravail	2026-05-06	Nina Couderchon	washingtonfrank
Bonjour Thier...	email	non répondu	0000-00-00		fonctionel SD/MM	Rhône-Alpes	pas_teletravail	2026-05-06	Nina Couderchon	washingtonfrank
Bonjour, Dans...	email	non répondu	0000-00-00		développeurs SAP ABAP	privas	teletravail	2026-02-23	Fanny SICARD	signeplus
Bonjour, Dans...	email	non répondu	0000-00-00		développeurs SAP ABAP	privas	teletravail	2026-02-23	Fanny SICARD	signeplus
Bonjour, Dans...	email	non répondu	0000-00-00		développeurs SAP ABAP	privas	teletravail	2026-02-23	Fanny SICARD	signeplus
Bonjour, Dans...	email	non répondu	0000-00-00		développeurs SAP ABAP	privas	teletravail	2026-02-23	Fanny SICARD	signeplus
Dear Thierry, ...	email	non répondu	0000-00-00		développeurs SAP ABAP	Rhône-Alpes	teletravail	2026-02-09	Nina Couderchon	washingtonfrank
Dear Thierry, ...	email	non répondu	0000-00-00		développeurs SAP ABAP	Rhône-Alpes	teletravail	2026-02-09	Nina Couderchon	washingtonfrank

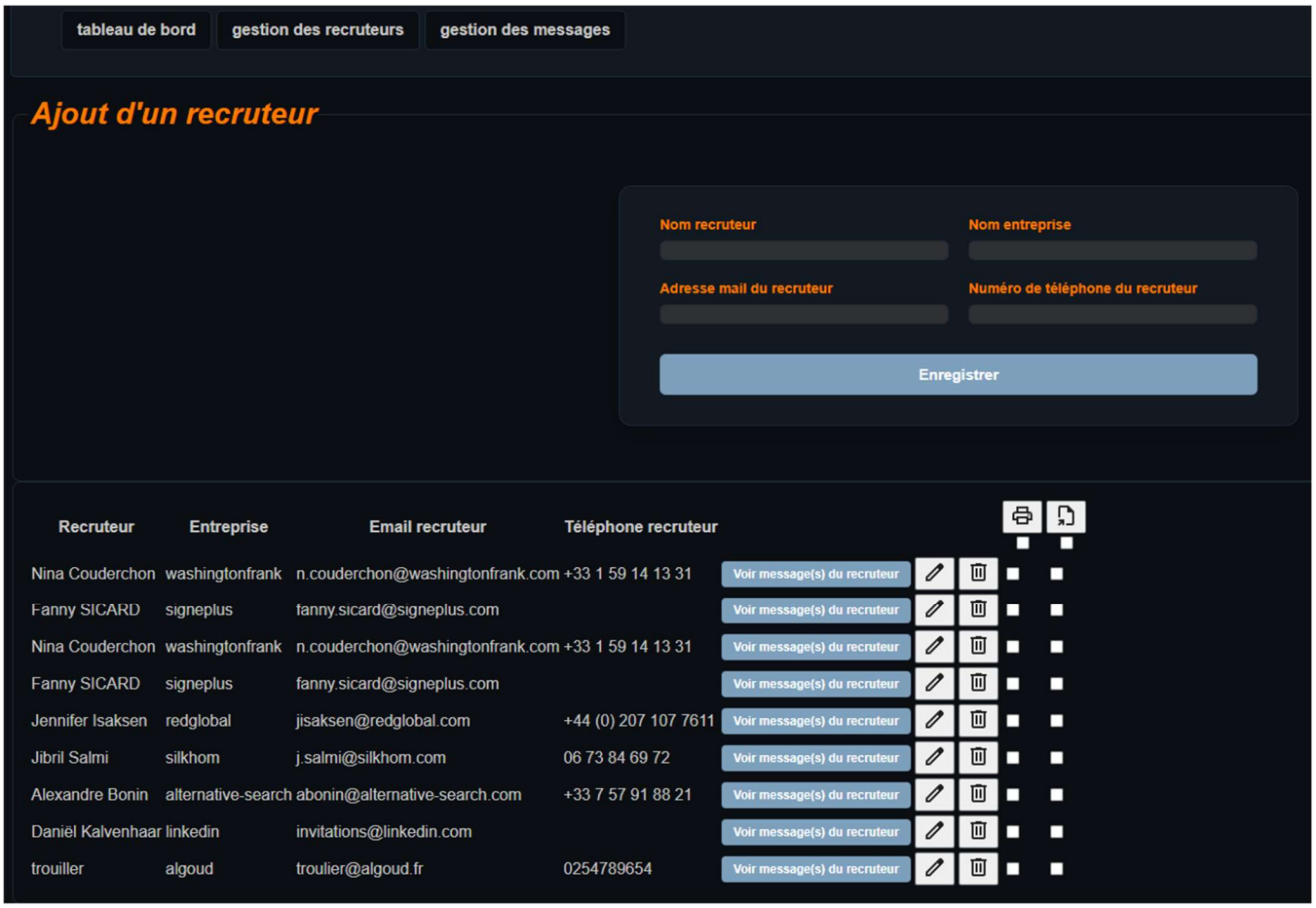
Voir plus

Chaque message dans la liste peut être modifié ou mis dans la corbeille.



Ci-dessus la capture d'écran présente le formulaire de modification d'un message.

Lorsqu'on clique sur le bouton 'Gestion des recruteurs', on obtient l'écran suivant :



Cet écran permet de visualiser la liste des recruteurs avec les possibilités de les modifier, les supprimer ou voir tous les messages qu'ils nous ont envoyés. On peut également saisir un nouveau recruteur.

ORGANISATION

TRDEV07 est une microentreprise. J'ai travaillé directement avec Monsieur Ricard.

Chaque jour nous faisions le point sur l'avancement de l'application. Lorsque j'avais fini de réaliser une fonctionnalité, Monsieur Ricard validait mon travail ou m'indiquait certains changements à réaliser.

J'ai ainsi progressé, étape par étape, dans la réalisation de l'application.

Pour faciliter la supervision de mon travail, Monsieur Ricard m'a demandé d'utiliser GitHub pour déposer mes fichiers.

J'ai travaillé sur site 4 jours par semaine. Les vendredis, j'étais en télétravail. Le fait d'avoir implémenté le projet sur GitHub m'a permis de travailler aussi bien de chez moi que dans l'entreprise.

De plus, la version finale est mémorisée sur le site et reste à disposition de Monsieur RICARD.

J’ai documenté mon travail sur le site de gestion des recruteurs en rédigeant une documentation utilisateur Documentions utilisateur.docx et une documentation technique (documentation tecnique.docx)

CONCLUSION

En conclusion j’ai pu apporter à entreprise une application fonctionnelle afin de lui permettre d’enregistrer des messages envoyés par les recruteurs.

Centrée sur la conception et le développement d'une application PHP, cette mission m'a permis de confronter mes connaissances théoriques aux exigences concrètes du monde professionnel. Sur le plan technique, j'ai pu consolider ma maîtrise du langage PHP et appréhender la structuration complète d'un projet web, ainsi que la création de l'interface utilisateur. Le fait d'évoluer en étroite collaboration avec M Ricard a également été une bonne expérience d'apprentissage.

Ce cadre de travail m'a poussé à faire preuve d'une grande autonomie, d'initiative et de réactivité face aux défis rencontrés au quotidien.

Ce stage m'a offert une vision réaliste du métier de développeur, en soulignant l'importance de bien analyser le besoin utilisateur pour proposer une solution adaptée.

En définitive, cette expérience valide pleinement les compétences acquises au cours de ma formation en services informatiques et me conforte dans mon projet professionnel.

ANNEXES

Les pages suivantes continuent à présenter les évolutions chronologiques apportées à l'application.

Pour l'implémentation du deuxième graphique, j'ai dû retravailler la div des graphiques afin de les aligner correctement.

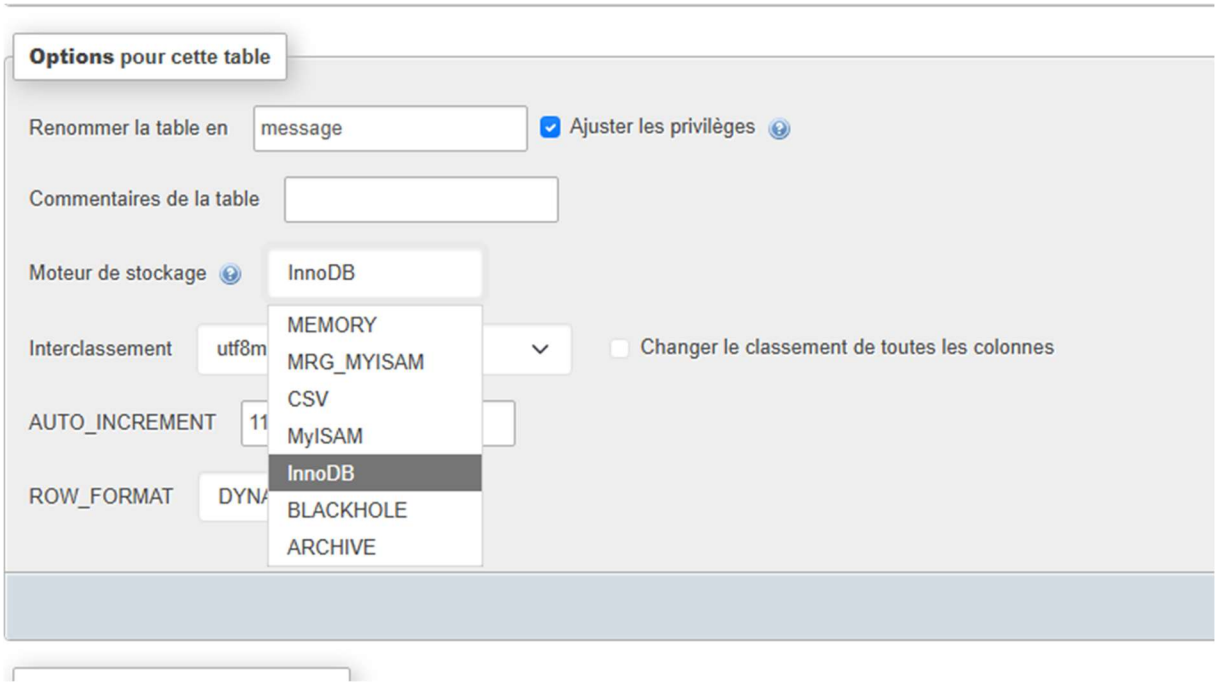
J'ai modifié mes requêtes dans la liste des recruteurs pour ajouter un INNER JOIN sur la table des

```php
if (isset($_POST['action']) && $_POST['action'] === 'modifyrecruteur') {
    $id_recruteur = $_POST['id'];
    $req = $bdd->prepare( query: "SELECT * FROM recruteur
                                  inner join entreprise on recruteur.id_entreprise = entreprise.id_entreprise
                                  WHERE id_recruteur = :id");
    $req->execute(['id' => $id_recruteur]);
    $recruteurData = $req->fetch( mode: PDO::FETCH_ASSOC);
```

entreprises. Cela m'a permis d'afficher le nom de l'entreprise de chaque recruteur directement dans la liste. J'ai également intégré cet INNER JOIN dans le traitement AJAX des recruteurs afin de préremplir correctement le champ "nom de l'entreprise" dans le formulaire

J'ai ensuite voulu faire en sorte que les entreprises qui ne possèdent aucun recruteur soient automatiquement supprimées. En cherchant pourquoi cela ne fonctionnait pas, j'ai constaté dans la base de données que je n'avais pas défini les clés étrangères (les colonnes n'étaient pas liées aux clés primaires correspondantes). J'ai donc configuré les contraintes de clés étrangères pour id_entreprise (liant la table recruteur à la table entreprise) et pour id_recruteur (liant la table message à la table recruteur)

J'ai également découvert que pour pouvoir utiliser les clés étrangères sous MySQL, il faut impérativement que les tables utilisent le moteur de stockage InnoDB. Or, par défaut, mes tables avaient été créées avec le moteur MyISAM.



Ajout des patterns pour la gestion des recruteurs, concernant l'e-mail et le numéro de téléphone, afin d'avoir de réelles valeurs d'e-mails et de numéros de téléphone.

```
<div class="form-group">
    <label for="email_recruteur">Adresse mail du recruteur</label>
    <input type="email" name="email_recruteur" id="email_recruteur" pattern="[a-z0-9._%+\-]+@[a-z0-9.\-]+\.[a-z]{2,}$" required>
</div>

<div class="form-group">
    <label for="tel_recruteur">Numéro de téléphone du recruteur</label>
    <input type="tel" name="tel_recruteur" id="tel_recruteur" pattern="[+ 0-9]{10-20}" required>
</div>
```

Ajout des boutons "nouveau message" et "vider le formulaire" dans la gestion des messages, pour pouvoir en créer un nouveau si on est en modification, et effacer le formulaire si on est en train d'en créer un.

```
function remplirLesChampsDuFormulaire(data) : void  {
    const sources : string[]  = ['telephone', 'linkedin', 'email', 'whatsapp'];
    const inputAutre : HTMLElement  = document.getElementById( elementId: 'autre

    if (inputAutre) {
        inputAutre.style.display = 'none';
        inputAutre.value = '';
    }

    if (sources.includes(data.source_message)) {
        const sourceRadio : HTMLElement  = document.getElementById(data.sourc
        if (sourceRadio) sourceRadio.checked = true;
    } else if (data.source_message) {
        const radioAutre : HTMLElement  = document.getElementById( elementId: 'a
        if (radioAutre) radioAutre.checked = true;
        if (inputAutre) {
            inputAutre.style.display = 'inline';
            inputAutre.value = data.source_message;
        }

    }

    if (btnvider) btnvider.style.display = 'none';
    if (btnnew)
        btnnew.style.display = 'inline-block';
}
```

```
    <div class="form-group full-width">
        <label for="message_reponse">Message de réponse</label>
        <textarea rows="6" name="message_reponsse" id="message_reponse"></textarea>
    </div>
    <button id="new_message" onclick="window.location.href='gestion_messages.php'" style="display: none"><img alt="new_message_icon" src="assets/new_message.svg"></butt
    <button type="reset" id="reset_form">vider formulaire</button>

    <input type="submit" name="valider" value="Enregistrer" class="btn-submit">
</div>
```

J’en ai profité pour modifier modificationmessage.js afin d'enlever un doublon dans le code.

J'ai modifié les légendes des formulaires afin qu'elles correspondent à l'action en cours, à savoir la modification ou l'ajout.

```
fieldset>
    <legend class="titre-form" id="ajout">ajout d'un recruteur</legend>
    <legend class="titre-form" id="modif" style="display: none">modification d'un recruteur</legend>
fieldset>
    <legend class="titre-form" id="ajout">Ajout d'un messages</legend>
    <legend class="titre-form" id="modif" style="display: none">modification d'un message</legend>
```

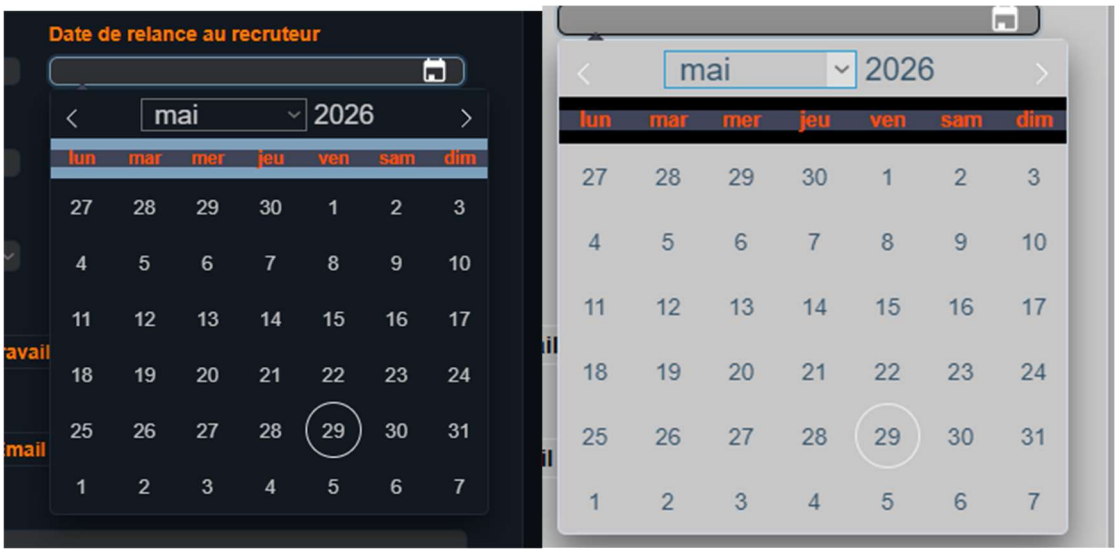
```
const titreajout :HTMLElement = document.getElementById( elementId: 'ajout');
const titremodif :HTMLElement = document.getElementById( elementId: 'modif');
if (nomr) nomr.value = data.nom_recruteur || '';
if (nome) nome.value = data.nom_entreprise || '';
if (emailr) emailr.value = data.email_recruteur || '';
if (telr) telr.value = data.No_telephone_recruteur || '';
const inputId :HTMLElement = document.getElementById( elementId: 'id_recruteur');
if (inputId) inputId.value = data.id_recruteur || '';
if (titreajout) titreajout.style.display = 'none';
if (titremodif) titremodif.style.display = 'inline-block';
```

J'ai aussi trouvé un moyen pour modifier l'apparence du calendrier des input de type date en installant une librairie JS.

```
</main>
<script src="https://cdn.jsdelivr.net/npm/flatpickr/dist/flatpickr.min.js"></script>
<script src="https://npmcdn.com/flatpickr/dist/l10n/fr.js"></script>
<script>
    // Applique le joli calendrier à tous les inputs de type "date"
    window.flatpickr("input[type=date]", {
        dateFormat: "Y-m-d", // Format pour ta base de données MySQL
        locale: "fr",
    });
</script>
```

```
.flatpickr-current-month .flatpickr-monthDropdown-months,
.flatpickr-current-month input.cur-year {
    color: var(--text-main) !important; /* Le texte s'adapte à ton thème
*/
    background: transparent !important;
}
span.flatpickr-weekday {
    background-color: transparent;!important;
    color: var(--text-sub) !important; /* Gris adapté à ton thème */
}
.flatpickr-current-month .flatpickr-monthDropdown-months option {
    background-color: var(--bg-container) !important;
    color: var(--text-main) !important;
}
.flatpickr-calendar {
    background-color: var(--bg-container) !important;
    border: 1px solid var(--border-color) !important;
    box-shadow: 0 10px 20px rgba(0,0,0,0.5) !important;
}

/* 2. La couleur du bandeau avec le Mois et l'Année */
.flatpickr-months .flatpickr-month {
    background-color: var(--bg-container) !important;
    color: var(--accent-blue) !important; /* J'utilise ta variable bleue,
remplace par #ff9900 si tu veux du orange */
}
/* 1. Empêcher les flèches de chevaucher Lundi et Dimanche (Force Brute)
*/
.flatpickr-months .flatpickr-prev-month,
.flatpickr-months .flatpickr-next-month {
    height: 30px !important;
    width: 30px !important;        /* On limite strictement la largeur */
    padding: 0 !important;         /* On tue la marge invisible de
Flatpickr */
    top: 5px !important;           /* On les remonte légèrement */
    display: flex !important;
    align-items: center !important;
    justify-content: center !important;
}

/* On force la barre des jours (lun, mar...) à passer AU-DESSUS des
flèches */
.flatpickr-weekdays {
    position: relative !important;
    z-index: 10 !important;
    background-color: var(--accent-blue) !important;
}
```

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/flatpickr/dist/flatpickr.min.css">
<link rel="stylesheet" type="text/css" href="https://npmcdn.com/flatpickr/dist/themes/dark.css">
```


Date de relance au recruteur
mai
2026
lun mar mer jeu ven sam dim
27 28 29 30 1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
1 2 3 4 5 6 7
mai
2026
lun mar mer jeu ven sam dim
27 28 29 30 1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
1 2 3 4 5 6 7

J’ai dû mettre la mention « !important » dans le style, car sinon les lignes se faisaient écraser par le fichier style.css par défaut de la bibliothèque. Les classes de style que j’ai modifiées proviennent de la bibliothèque du calendrier.

J’ai essayé de trouver pourquoi, lors de la première arrivée sur la page d'accueil, on ne pouvait pas modifier un message. Je me suis aperçu que c’était une condition if dans index qui empêchait cela ; tout ce qu'il y avait à l'intérieur, et même l'appel à chargerMessagePourModification juste après, ne servait à rien. C'est la fonction chargerMessage, appelée à la fin, qui suffit à charger les données.

```
Show usages   kamisoris *
function lierEvenementsBoutons() : void  {
    document.querySelectorAll( selectors: '.modify').forEach( callbackfn: button : Element  => {
        button.addEventListener( type: 'click',  listener: function() : void  {
            // Si on est sur l'index, on redirige vers gestion_messages.php
            alert("ici")

            //if (window.location.pathname.includes('index.php')) {
                window.location.href = 'gestion_messages.php?action=modify&id=' + this.value;

                // return;
            //}

            // Sinon on charge les données directement
            //chargerMessagePourModification(this.value);
        });
    });

    document.querySelectorAll( selectors: '.delete').forEach( callbackfn: button : Element  => {
```

J’ai ensuite fait le nécessaire pour que l'on puisse sélectionner les recruteurs afin de les imprimer ou de les exporter. Pour cela, j'ai ajouté deux boutons distincts : un pour imprimer et un autre pour exporter.

```
<th></th>
<th><button id="btn_print_liste"><img alt="impression" src="assets/imprimante.svg"></button><br>
        <label>
            <input type="checkbox" id="check_all_print">
        </label>
</th>
<th>
    <button><img alt="extraction_email" src="assets/export.svg"></button><br>
        <label>
            <input type="checkbox" id="check_all_extract_email">
        </label>
```

```
<td><label>
        <input type="checkbox" class="impression">
    </label></td>
<td><label>
        <input type="checkbox" class="extract_email">
    </label></td>
```

```
new *
document.addEventListener( type: 'DOMContentLoaded', listener: function() : void {
    const allprint : HTMLElement = document.getElementById( elementId: "check_all_print")
    const print : NodeListOf<Element> = document.querySelectorAll( selectors: ".impression")
    const extractall : HTMLElement = document.getElementById( elementId: "check_all_extract_email")
    const extract : NodeListOf<Element> = document.querySelectorAll( selectors: ".extract_email")
    if (allprint)
    {
        allprint.addEventListener( type: 'change', listener: function () : void
        {
            print.forEach( callbackfn: function (checkbox : Element ) : void
            {
                checkbox.checked = allprint.checked;
            });
        });
        print.forEach( callbackfn: function (checkbox : Element ) : void
            {
                checkbox.addEventListener( type: 'change', listener: function () : void
                {
                    const checkcount : number = document.querySelectorAll( selectors: ".impression:checked").length;
                    allprint.checked = (checkcount === print.length);
                });
            });
    }
```

J'ai par la suite mis en place le bouton d'impression à l'aide d'une iframe. Cela offre la possibilité d'imprimer uniquement les éléments sélectionnés : au lieu de générer une page web secondaire pour lancer l'impression, on utilise une iframe invisible. Cela

permet de n'afficher que l'aperçu avant impression du navigateur, sans laisser de page

```
if (print && extract)
{
    const printer : HTMLElement = document.getElementById( elementId: "btn_print_liste");
    printer.addEventListener( type: 'click', listener: function(event : PointerEvent ) : void
    {
        event.preventDefault();
        printcheck = document.querySelectorAll( selectors: '.impression:checked')
        let listerecruteur : any[] = [];
        printcheck.forEach( callbackfn: function (box : Element ) : void
        {
            // on insère les valeur des case dans info
            let infos = box.value.split('-');
            //on crée les ligne du tableau impression
            let lignetableau : string = "<tr>"+
                "<td>"+infos[0]+"</td>"+
                "<td>"+infos[2]+"</td>"+
                "<td>"+infos[3]+"</td>"+
                "<td>"+infos[1]+"</td>"+
                "</tr>";
            //on inject dans lsiterecruteur les élément de lignetableau
            listerecruteur.push(lignetableau);
        });
        if (listerecruteur.length>0){
            //évite de séparer avec des virgule apr défault on ne veut  séparer avec rien on vuteut coller les élément pour le tableau
```

HTML visible en arrière-plan.

```
setTimeout( handler: function () : void
{
    //focus pour éviter d'impriemr la apge entière suite à inframe invisible
    iframe.contentWindow.focus();
    iframe.contentWindow.print();
    setTimeout( handler: function () : void {
        document.body.removeChild(iframe);
    }, timeout: 1000)
}, timeout: 250);
}else{
    alert("Veuillez sélectionner au moins un recruteur à imrimer")
}
```

J'ai ensuite corrigé un problème qui faisait que je n'arrivais pas à modifier un recruteur depuis la page

```
nter.addEventListener( type: 'click', listener: function(event : PointerEvent ) : void
    "th { background-color: #f2f2f2; }";
</div>
<?php
 include 'liste_message.php';}
?>
    <script><?php include 'script/modificationmessage.js'; ?></script>
    <script><?php include 'script/modificationrecruteur.js'; ?></script>
```

```
const baliseStyle : HTMLStyleElement = iframeDoc.createElement( tagName: 'style');
baliseStyle.innerHTML = styleCSS;
iframeDoc.head.appendChild(baliseStyle);
iframeDoc.body.innerHTML =
    "<table>" +
    "<thead>" +
    "<tr>" +
    "<th>Nom</th>" +
    "<th>Email</th>" +
    "<th>Numéro téléphone</th>" +
    "<th>Nom entreprise</th>" +
    "</tr>" +
    "</thead>" +
    "<tbody>" +
```

d'accueil (index). Le problème venait du fait que j'incluais deux fichiers JS au sein d'une seule et même balise <script>.

J’ai ensuite développé le bouton permettant d'extraire les e-mails dans un fichier texte. Cela reprend la même logique que pour l'impression, sauf qu'au lieu d’utiliser une iframe (qui sert à imprimer), on utilise un Blob. Cet objet, natif en JavaScript, permet de créer un fichier virtuel que l'utilisateur peut ensuite télécharger.

```
    });
    const extracter :HTMLElement = document.getElementById( elementId: "btn_extract_mail");
    extracter.addEventListener( type: 'click', listener: function (event :PointerEvent ) :void
        {
            event.preventDefault();
        extractercheck = document.querySelectorAll( selectors: '.extract_email:checked');
        let listeemail :any[] = [];
        extractercheck.forEach( callbackfn: function (box :Element ) :void {
            listeemail.push(box.value);
        });
        if (listeemail.length >0)
        {
            //le fichier qui contiendra la lsite des mail
            const fichiermail :string = listeemail.join('\n');
            //fichier virtuelle de type text/plain(.txt)
            const blob :Blob = new Blob( blobParts: [fichiermail], options: {type: 'text/plain;charset=utf-8'});
            //lien de téléchargement invisible
            const extractionfichiermail :HTMLAnchorElement = document.createElement( tagName: 'a');
            extractionfichiermail.href = URL.createObjectURL(blob);
            extractionfichiermail.download = 'extraction_emails_recruteur.txt';
            document.body.appendChild(extractionfichiermail);
            extractionfichiermail.click();
            document.body.removeChild(extractionfichiermail);
            URL.revokeObjectURL(extractionfichiermail.href);
        }else {
            alert("Veuillez sélectionner au moins un recruteur pour extraire les adresse  mail");
        }

        })

}
```

J’ai rédigé une documentation utilisateur et commencé une documentation technique. La documentation utilisateur sert à guider l'utilisateur lors de sa navigation sur le site. La documentation technique, quant à elle, permet à l’entreprise où j’effectue mon stage de comprendre la structure du site afin de pouvoir le modifier ou le réinstaller, par exemple en cas de panne de disque dur ou pour toute autre raison.